

VERIQO Incident Response Procedure

Status: written, not yet exercised. This procedure has not been run in a real incident or a tabletop exercise. It should be walked through by whoever will operate a deployment *before* it is needed, and revised from what that exercise reveals.

1. Scope

Covers incidents affecting a VERIQO deployment: suspected compromise, data loss, integrity failure, and availability loss.

The defining characteristic of an incident in an evidence system: the damage is not only operational. If the integrity of the record is in question, every decision that record supports is in question, including decisions taken months ago. Integrity incidents therefore outrank availability incidents here, which inverts the usual priority order.

2. Severity

Sev	Definition	Examples
SEV-1	Record integrity is in question, or evidence may be lost.	<code>ledger_hash_chain</code> verification fails; a WAL is corrupt beyond the recoverable tail; replay diverges across multiple cases; suspected unauthorized write access.
SEV-2	Confidentiality or authorization boundary may be breached.	Suspected credential compromise; a caller reached another tenant's data; signing key material possibly exposed.
SEV-3	Availability loss, integrity intact.	Service down; <code>/readyz</code> failing; disk exhausted.
SEV-4	Degraded or anomalous, no confirmed impact.	Elevated <code>evidence_verification_failures</code> ; a single case's replay diverges.

A single evidence item failing verification is SEV-4 until investigated; a ledger chain failure is SEV-1 immediately. The difference is that one item may be damaged, whereas a broken ledger means the record as a whole does not hang together.

3. First response — preserve before you fix

The instinct to restart the service and restore from backup is **wrong here** if integrity is in question. Doing so destroys the evidence of what happened.

Before any remediation, in order:

1. **Stop writes.** Take the deployment out of load-balancer rotation (`/readyz` failing is sufficient) rather than killing the process.
2. **Copy the WAL directory as-is**, byte for byte, to separate storage. Do not use `Backup` for this — copy the raw directory including any corrupt tail. The corruption is the evidence.
3. **Capture** the process logs, `GET /v1/metrics`, `/readyz` output, deployment configuration (without secrets), and the binary's version identity.

4. **Record the wall-clock time** of each step. VERIQO's own ticks are logical, so your incident timeline must come from your side.
 5. Only then begin diagnosis.
-

4. Playbooks

4.1 Ledger chain verification fails (SEV-1)

1. Preserve as in §3. Do not restart.
2. Run the standalone verifier against recently exported packages for affected cases. A package exported *before* the incident that still verifies establishes a known-good point in time.
3. Determine the first bad record: chain verification identifies where re-derivation stops matching.
4. Establish whether the cause is corruption (disk, truncation) or alteration (a write that should not have happened). Corruption usually damages a contiguous region; targeted alteration usually does not.
5. **Notify affected customers.** A record whose integrity cannot be established must not be presented as if it could be. This is not optional and should not wait for full root cause.
6. Restore from the most recent backup that verifies cleanly, and determine what was lost between that backup and the incident.

4.2 Evidence verification failure (SEV-4 → escalate)

1. `POST /v1/evidence/{id}/verify` on the affected item; check whether the manifest hash or the custody chain failed.
2. Check whether other items fail. **One item = likely item-level damage; many items = escalate to SEV-1.**
3. Check `custody_chain_failures` and `evidence_verification_failures` in metrics for the trend.
4. Preserve, then investigate. Do not re-submit a "corrected" version of the item to make the error go away — that destroys the record of what happened and is itself an integrity problem.

4.3 Replay divergence

Follow the Replay Specification §6. Single-case divergence points at that case's evidence; all-case divergence points at the deployment (usually a software version change or a bad restore).

4.4 Suspected credential or key compromise (SEV-2)

1. **Revoke the affected signing key** through the key manager. Revocation is retroactive — signatures made while the key was active will subsequently fail verification. This is intended: a possibly-compromised key must not continue to confer authenticity.
2. Rotate to a new key. Re-issue your trusted-key registry to every party who verifies your packages, **through the same out-of-band channel you used originally.**
3. Revoke and reissue JWT signing material and API keys.
4. Notify counterparties holding packages signed by the revoked key: their verification will now report the key as revoked, which they must be able to interpret.

4.5 Cross-tenant access (SEV-2)

1. Preserve logs immediately; this is the one incident type where the request log is the primary artifact.

2. Determine whether the deployment ran with JWT authentication configured. If it did not, tenant IDs were caller-supplied by design — this is a deployment misconfiguration, not a control failure.
3. If JWT was configured, this indicates a genuine authorization defect. Preserve everything and report it to VERIQO engineering with the request log.
4. Notify both tenants.

4.6 Service down (SEV-3)

Check `/livez` (process alive) and `/readyz` (dependencies fit). A process alive but not ready usually means the durable store is closed or failed to open. Check disk space first — a full disk stops WAL writes, and a write that cannot be made durable is correctly refused rather than silently accepted.

Restart is acceptable here **only** once you have confirmed integrity is not in question.

5. Recovery

Restoring is described in `VERIQO_BACKUP_AND_RESTORE_PROCEDURE.md`. Integrity-specific requirements:

- Verify the restored store before returning it to service: run replay on several cases and confirm convergence.
 - Determine and **document the recovery gap** — what was accepted between the backup and the incident and is now absent. Customers whose evidence falls in that window must be told; silently serving a store that is missing accepted writes is the worst available outcome.
 - Keep the preserved pre-recovery copy. Do not overwrite it with the recovered store.
-

6. Communication

Notify customers when: record integrity is in question; evidence may be lost; a tenant boundary may have been crossed; a signing key is revoked; or there is a recovery gap.

State plainly what is known, what is not yet known, which cases are affected, and whether previously exported dossier packages remain valid (usually they do — they are self-contained and verify independently of the deployment's current state, which is a genuinely useful thing to be able to say during an incident).

Do not state that integrity is intact until it has been verified. "We are verifying" is an acceptable interim message; a premature all-clear is not.

7. After the incident

Write up: timeline in wall-clock time, what failed, what was preserved, what was lost, what customers were told and when, and what changes follow. Where the incident revealed a defect in VERIQO itself rather than in the deployment, report it with the preserved artifacts — integrity defects are the highest-priority class of bug in this system.

Then revise this document. Its first real exercise will show what it got wrong.